Any Ruby object can be used as a matcher as long as it implements a minimal set of methods. This protocol is so simple that you can even build up a matcher object in IRB. Over the next few code examples, we're going to do just that.

Back in your IRB session, type in the following code (if you're starting a new session, don't forget to rerun the setup code):

```
>> matcher = Object.new
=> #<Object:0x007fe2e213ea58>
```

Let's see how RSpec tries to use this object when we pass it as a matcher:

```
>> expect(1).to matcher
NoMethodError: undefined method `matches?' for #<Object:0x007feff9326f18>
    « backtrace truncated »
```

This expectation has triggered a NoMethodError exception. RSpec expects every matcher to implement a matches? method, which takes an object and returns true if the object matches (and false otherwise). Let's define that now, and try the matcher again:

```
>> def matcher.matches?(actual)
>>   actual == 1
>> end
=> :matches?
>> expect(1).to matcher
=> true
```

Success! What about when the match fails? Let's try matching against 2 now:

```
>> expect(2).to matcher
NoMethodError: undefined method `failure_message' for                    ↩
#<Object:0x007fe2e213ea58>
    « backtrace truncated »
```

When the provided object does not match, RSpec calls the matcher's failure_message method to get an appropriate message to display to the user. Let's define that now:

```
>> def matcher.failure_message
>>   'expected object to equal 1'
>> end
=> :failure_message
>> expect(2).to matcher
RSpec::Expectations::ExpectationNotMetError: expected object to equal 1
    « backtrace truncated »
```

With that method in place, RSpec now throws an ExpectationNotMetError containing our message.